

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour Questions: 5 Total Marks: 50

Annexure A

Scenario Based Assignment

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CSA17 — ARTIFICIAL INTELLIGENCE

Assessment Tool 2 — Scenario Based Assignment: Answers

CO2: Search and game-solving techniques — Greedy Search, A*, CSP, Minimax, Alpha-Beta pruning, heuristic functions

Scenario 1. Emergency Medicine Delivery Drones in a Flood-Affected Region

A drone has a straight-line-distance heuristic, partial real-time updates about blocked paths, and variable terrain/weather-dependent movement costs, and must choose routes with minimum delay and risk under urgency.

Task 1 — Greedy Best-First Search Behaviour, Strengths and Weaknesses Under Uncertainty

Greedy Best-First Search would always expand toward whichever next waypoint has the smallest straight-line distance to the destination, using only $h(n)$ and completely ignoring the cost or risk already accumulated along the way ($g(n)$ plays no role at all in its decisions).

- Strength: extremely fast decisions with very little computation per step — directly valuable here since the drone must "quickly decide routes with minimum delay," and Greedy never has to reconsider cost history.
- Strength: simple to implement and reason about, which matters for an onboard system with limited compute.
- Weakness under uncertainty: because it is blind to real terrain/weather cost, it can confidently head toward a waypoint that looks close in a straight line but sits behind a flooded, now-impassable road, or across dangerous weather — the heuristic has no way to represent a road that "suddenly becomes inaccessible."
- Weakness: gives no optimality or safety guarantee — it may pick a geometrically short but practically costly/dangerous route over a longer but safer, cheaper one, which is a serious concern when the cargo is emergency medicine.

Task 2 — How A* Handles the Same Situation

A* evaluates each candidate node with $f(n) = g(n) + h(n)$, where $g(n)$ is the actual accumulated cost/risk incurred so far (incorporating the real, variable terrain and weather costs) and $h(n)$ is the same straight-line-distance estimate used by Greedy.

- Because $g(n)$ is included, A* will not blindly commit to a path just because it looks short in a straight line — if that path has already proven expensive or risky (bad terrain, poor weather) it competes fairly against alternative routes rather than being chosen by heuristic appearance alone.
- A* provably finds the optimal (minimum total cost/risk) path from start to goal, provided the straight-line-distance heuristic is admissible — which it naturally is here, since a straight line is always the shortest possible distance and can never overestimate the true travel distance.
- This balance of "cost so far" and "estimated cost to go" is exactly what lets A* prefer a genuinely safer, cheaper overall route even when it is not the one that looks most direct at any single step.

Task 3 — Impact of Heuristic Accuracy on Both Algorithms

- For Greedy Best-First Search, heuristic accuracy is everything — since $h(n)$ is the only signal driving its decisions, a heuristic that poorly reflects real difficulty (e.g., straight-line distance across a lake, mountain, or flood zone) directly and immediately causes bad routing, with no cost-tracking safety net to catch the mistake.

- For A*, heuristic accuracy mainly affects efficiency, not correctness: a weaker (more conservative/less accurate) admissible heuristic simply causes A* to explore more nodes and take longer, but as long as it stays admissible (never overestimates), the final route A* returns is still guaranteed optimal.
- The only way heuristic accuracy breaks A*'s correctness is if it becomes inadmissible (overestimates true remaining cost) — then A* can be misled the same way Greedy always is, and may return a non-optimal route.

Task 4 — Effect of Dynamic Changes (Blocked Paths) on Search Performance

- Both algorithms, in their classic textbook form, assume a fixed graph computed once — a genuinely dynamic environment where paths become blocked mid-flight breaks that assumption for either of them and requires an online, re-planning layer on top of whichever base algorithm is used.
- Greedy is cheap to re-run from the drone's current position whenever a blockage is reported, since it tracks very little state — but each fresh re-plan is again only as good as the (still cost-blind) heuristic, so it remains vulnerable to the same class of mistakes every time.
- A* is more expensive to re-run per re-plan (larger frontier/priority-queue bookkeeping), but each re-plan continues to produce a cost-aware, closer-to-optimal route given the newly updated information — a real-time updated blockage simply needs its edge cost effectively set to "impassable" (infinite) and the search re-run from the drone's current location.
- In a highly dynamic environment like a flood zone, frequent re-planning is unavoidable regardless of algorithm choice — the real design question is how efficiently each algorithm can be re-triggered and how much of the previous search can be reused rather than recomputed from scratch.

Task 5 — Recommendation and Justification

The most suitable approach is an **online, incrementally re-planned A* search** (optionally a weighted/bounded variant of A* that slightly inflates the heuristic to trade a small amount of optimality for faster decisions under the tightest time pressure), rather than plain Greedy Best-First Search.

- Real-time constraints: A* re-planning is more expensive per step than Greedy, but a weighted or bounded variant keeps it fast enough for the drone's urgency requirement while still consulting actual accumulated cost, unlike Greedy which is fast purely by ignoring information it should not ignore in a safety-critical delivery.
- Optimality: A* (with an admissible heuristic like straight-line distance) gives a meaningfully better cost/risk guarantee than Greedy, which has none at all.
- Safety: since this is emergency medicine delivery, a "fast but wrong/dangerous" route (a real risk with pure Greedy) is not acceptable — cost-aware A*-based routing, re-triggered online whenever new blocked-path updates arrive, is the well-justified choice balancing speed, correctness, and safety together.

Scenario 2. Smart City Traffic Signal Optimization

An AI system must continuously adjust signal timings across hundreds of intersections to minimize total waiting time, fuel consumption, and congestion, over an extremely large, dynamically-changing search space that risks getting stuck in locally optimal solutions.

Task 1 — Formulating as an Optimization Problem and Why Traditional Search Is Inefficient

This is naturally an **optimization problem**, not a path-finding search problem: a state is one complete configuration of signal-timing settings across every intersection; the objective function is a weighted combination of total waiting time, fuel consumption, and congestion (to be minimized); and only the quality of the final

configuration matters — there is no meaningful notion of a "path" from a start state to a goal state the way there is in route-finding.

- Traditional path-search algorithms (BFS, DFS, UCS, A*) are built around finding a path through a state graph to a specific goal test — but here there is no single discrete goal state to test for, only "which configuration scores best," so the entire path-finding framing is a poor conceptual fit.
- Even setting that aside, the combinatorial space of possible timing configurations across hundreds of intersections is astronomically large (each intersection has many possible timing/phase settings, multiplied across all intersections), making an exhaustive or even a systematic graph-traversal search computationally intractable at city scale.

Task 2 — How Hill Climbing Would Work Here, and Its Limitations

Hill Climbing would start from some initial signal-timing configuration and repeatedly try small local tweaks (e.g., adjust one intersection's green-light duration slightly), keeping any change that improves the overall objective score, and stopping when no small local change helps any further.

- Limitation: it only ever looks at neighbouring configurations one small step away, so it can settle at a configuration that is locally good but far from the best achievable city-wide arrangement.
- Limitation: it has no mechanism to escape a plateau (nearby changes make no measurable difference) or a ridge (the only real improvement requires changing several intersections together, not one at a time).
- Limitation: its outcome is highly sensitive to the starting configuration, and it gives no guarantee whatsoever of finding the best possible city-wide signal-timing scheme — only "a nearby better one" than wherever it started.

Task 3 — Local Maxima, Plateaus, and Ridges — Real-World Interpretation

- Local maximum: a configuration where every small individual tweak makes things worse, yet a coordinated set of changes across several intersections together would be much better — e.g., creating a "green wave" along a busy arterial road requires adjusting multiple signals' offsets together; changing just one of them in isolation might look like it hurts overall waiting time, hiding the real improvement from a one-at-a-time search.
- Plateau: a stretch of essentially equally-good configurations — e.g., minor timing tweaks at a quiet residential intersection barely change city-wide congestion at all, so Hill Climbing wanders with no clear improving direction and wastes computation there.
- Ridge: a narrow, improving direction that only appears when two or more variables change together — e.g., real congestion relief may require simultaneously adjusting two adjacent intersections' offsets in a coordinated way, which a strategy that only changes one variable at a time (as basic Hill Climbing does) may never discover, since neither change alone reveals the benefit.

Task 4 — How Simulated Annealing or Genetic Algorithms Overcome These Limitations

- Simulated Annealing occasionally accepts a worse move, governed by a "temperature" parameter that gradually decreases over time — this lets it escape local maxima and cross plateaus by tolerating short-term dips in performance in exchange for exploring more of the space; as the temperature cools, it increasingly settles into exploiting the best region it has found, typically converging to a substantially stronger solution than plain Hill Climbing.
- Genetic Algorithms maintain an entire population of candidate signal-timing configurations at once (not just a single current configuration), combine ("crossover") promising parts of different good configurations, and introduce random mutations — this parallel, population-based exploration is naturally much better suited to

discovering coordinated, multi-intersection improvements (the ridge case above) and to avoiding any single configuration's local trap, since many different regions of the search space are being explored simultaneously.

Task 5 — Recommended Approach and Justification (Scalability and Adaptability)

The recommended approach is a **Genetic Algorithm**, optionally hybridized with Simulated Annealing for local refinement of its best individuals.

- Scalability: a GA's population-based search parallelizes naturally across many candidate solutions and many intersections, and its crossover operator specifically excels at combining good partial solutions from different parts of the city — a much better structural fit than a single-point search when hundreds of intersections must be coordinated together.
- Adaptability: traffic patterns change dynamically throughout the day (rush hour vs. night, accidents, weather); a GA's population can be periodically re-evolved using updated real-time sensor data to continuously adapt, whereas Hill Climbing would need a full restart each time and would simply re-trap itself in a new local maximum.
- Simulated Annealing remains a reasonable, lighter-weight option where compute resources are constrained, and works well as a complementary local-refinement step polishing the best individuals a GA has already found — combining both methods' strengths rather than treating them as strictly either/or.

Scenario 3. Autonomous Mars Rover Exploration

A rover must navigate unknown Martian terrain, avoid hazards, operate with limited sensing, and make decisions without a complete map, continually updating its knowledge as it explores.

Task 1 — Why This Requires an Online Search Agent, Not Offline Search

Offline search (e.g., running plain A* or UCS once) requires the entire environment to be known in advance so the whole search graph can be built and a full plan computed before any action is taken. The rover has no such complete map of the Martian terrain — there is, quite literally, no full graph available to search offline.

- An online search agent instead interleaves acting and sensing: it takes one exploratory action, observes what that reveals, updates its internal knowledge, and re-plans from its new position — the only approach that is even possible when the environment must be discovered through actual exploration rather than known in advance.

Task 2 — Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover can only perceive a limited local area around itself (its sensing radius), so at any moment it must decide with incomplete information — a hazard just beyond current sensing range (a hidden crater, loose rocks) cannot be anticipated, meaning every decision carries genuine risk under uncertainty.
- Dynamic elements: although the physical terrain is largely static over short timescales, the rover's own knowledge state is constantly changing as it explores, and real conditions (settling dust, small rockslides, shifting shadows affecting what sensors can reliably detect) can alter what is safe or even visible over time — so previously sensed information cannot always be trusted indefinitely.
- Together, these mean the rover cannot simply commit to one long precomputed plan; a plan that looked fine given current knowledge may need revising the moment new sensing reveals that knowledge was incomplete or has become stale.

Task 3 — How the Rover Builds and Updates Its Internal Model

- The rover maintains an internal map (the core structure of a model-based agent): cells or regions it has directly sensed are marked safe, hazardous, or still unknown, based on camera, LIDAR, or other range-sensor readings.
- As it moves, newly sensed areas are added to or correct this map, and previously "unknown" cells adjacent to its current position become known once actually sensed, progressively narrowing uncertainty.
- This internal model is exactly what a local search algorithm (e.g., an online UCS/A* re-run from the rover's current position at each step) operates over — without it, there would be no state graph to search at all, since raw, momentary percepts alone retain no history.
- Whenever something embedded in the model turns out to be wrong (a cell assumed traversable turns out to be a hidden crater once sensed up close), the model is corrected immediately and the local plan is revised accordingly.

Task 4 — Comparing Online Search with Uninformed and Informed Offline Strategies

- Uninformed offline search (BFS/DFS/UCS) requires a complete, known state graph in advance — infeasible here since no full Martian terrain map exists; even if one somehow did, blindly executing a long precomputed plan would ignore any hazard discovered along the way.
- Informed offline search (A* with, say, straight-line distance to the target) has the same fundamental problem: it still needs the whole graph and cost structure known upfront before it can search at all, and cannot adapt mid-plan to anything the map failed to capture.
- Online search keeps the strengths of informed search (it can still use a heuristic and cost-based local planning at each step) but only ever plans over the currently-known portion of the map, executes one step, and re-senses/re-plans — trading some efficiency (it may occasionally have to backtrack from a dead end a fully-informed offline planner would have avoided) for the ability to function at all in a genuinely unknown, partially observable environment.

Task 5 — Justifying the Most Suitable Approach (Uncertainty, Adaptability, Safety)

The most suitable approach is a **model-based, goal-based online search agent** — an adaptive/online variant of A* or UCS layered over the rover's continuously updated internal map, as described above.

- Uncertainty: only an online approach can function correctly at all here, since the rover fundamentally cannot know the full terrain map in advance.
- Adaptability: the sense → plan → act → update loop naturally folds in every new piece of information the rover encounters, letting its route continuously improve or correct itself as exploration proceeds.
- Safety: by only ever committing to short local plans built on currently sensed, verified information — rather than a long precomputed path resting on unverified assumptions — the rover avoids confidently driving into a hazard that a stale or incomplete map might have missed, which is critical for an irreplaceable piece of hardware with no possibility of on-site repair.

Scenario 4. AI-Generated University Exam Timetables

An automated timetabling system must satisfy no student overlaps, limited exam halls, subject-ordering requirements, faculty availability, and special accommodations, with complexity growing as courses and students increase.

Task 1 — Formulating the Problem as a Constraint Satisfaction Problem (CSP)

Framed as a CSP, the problem has: **variables** — one for each exam that needs scheduling; **domains** — for each exam, the set of valid (time slot, exam hall) pairs it could be assigned; and **constraints** — the rules listed in the scenario that any complete, valid assignment must simultaneously satisfy.

Task 2 — Defining Variables, Domains, and Constraints, with Explanation

- Variables: Exam_CS101, Exam_CS102, Exam_MATH201, ... — one variable per course requiring an exam.
- Domain of each exam-variable: every valid (time slot, hall) combination on the exam calendar, e.g. {(Mon 9am, Hall A), (Mon 9am, Hall B), (Mon 2pm, Hall A), ...}.
- No-overlap constraint: for any two exams that share at least one student, their assigned time slots must differ — a pairwise (binary) constraint applied across every such pair of exam-variables.
- Limited-hall / capacity constraint: for any given time slot, the number of exams scheduled simultaneously cannot exceed the number of available halls, and the students assigned to any one hall cannot exceed its seating capacity — a resource-style constraint shared across all exams in that slot.
- Precedence constraint: for specified subject pairs, one exam's assigned slot must occur strictly before another's (e.g., a prerequisite course's exam before its follow-on course's).
- Faculty-availability constraint: the invigilator(s) assigned to an exam must be marked available at that exam's assigned time slot — a unary constraint tying an exam's slot choice to a precomputed faculty availability table.
- Special-accommodation constraint: students needing extended time or a separate room restrict which domain values are valid for their specific exams (e.g., only rooms flagged "accessible," or slots with an extended-duration allowance).

Task 3 — How Backtracking Search Works in This Scenario

Backtracking assigns exams to (slot, hall) values one at a time, in some chosen order, checking after every assignment whether it violates any constraint against exams already scheduled (no shared-student clash, hall capacity respected, faculty free, precedence honoured, accommodation needs met).

- If a partial assignment violates a constraint, the algorithm backtracks — undoes the most recent assignment and tries a different value from that exam's domain (or an earlier exam's, once the current one's domain is exhausted) — continuing until either a complete, valid timetable is found, or every possibility is exhausted, meaning (as formulated) no valid timetable exists with the current halls/slots and additional resources would be needed.
- Heuristics such as Minimum Remaining Values (schedule the most constrained, hardest-to-place exam first — e.g., large courses overlapping with many others) and Least Constraining Value (try the option that rules out the fewest possibilities for remaining exams first) are typically layered on top to make backtracking practical at real university scale.

Task 4 — How Constraint Propagation (e.g., Forward Checking) Improves Efficiency

Forward checking, immediately after an exam is assigned a (slot, hall), removes the now-invalid options from the domains of every as-yet-unassigned exam that shares a constraint with it — for example, removing that same slot from the domain of every other exam sharing a student with the one just scheduled, and reducing the remaining hall-capacity count for that slot.

- This lets the algorithm detect a dead end far earlier: if some unassigned exam's domain becomes empty as a direct consequence of an earlier choice, backtracking can happen immediately, instead of continuing to assign several more exams first and only discovering the failure much later, deep in the search tree.

- More advanced propagation (e.g., full arc-consistency such as AC-3) can catch even more failures earlier still, at some extra per-step computational cost, and becomes increasingly worthwhile as the number of courses and students — and therefore the density of constraints — grows.

Task 5 — Real-World Challenges and the Best Strategy for Scalability

- Real-world challenge: the number of exam pairs sharing at least one student grows roughly quadratically as enrollment and course overlap increase, so the constraint graph becomes extremely dense at university scale.
- Real-world challenge: special accommodations and faculty-availability rules add many further unary/binary restrictions, and limited exam halls impose a genuinely tight global resource constraint (not just pairwise clashes) that can cause naive orderings to hit dead ends very late in the search.

The best strategy for scalability is **backtracking search combined with strong constraint propagation (forward checking or arc-consistency) and variable/value ordering heuristics** (Minimum Remaining Values to tackle the most constrained exams first, Least Constraining Value to keep options open for the rest). For very large universities, this is typically further combined with problem decomposition — e.g., scheduling department-by-department, or splitting the exam period into largely independent blocks — since pure exhaustive backtracking without propagation and good ordering simply does not scale to real university enrollment sizes.

Scenario 5. Strategic Game AI for Real-Time Decision Making (Minimax and Alpha-Beta Pruning)

A gaming company's AI opponent must compete against human players within strict time limits, evaluating complex game states from an extremely large, computationally expensive decision tree.

Task 1 — How the Minimax Algorithm Models This Problem

The game is modeled as a tree of alternating turns: nodes are game states and edges are possible moves, with a maximizing layer (the AI's turn — picks the move leading to the best outcome for itself) alternating with a minimizing layer (the opponent's turn — assumed to always pick the move worst for the AI). Minimax recursively computes, for every state, the best outcome achievable under the assumption of perfect, worst-case-for-you play from the opponent at every subsequent step, and the AI ultimately chooses whichever root-level move guarantees the highest value under that worst-case assumption.

Task 2 — Computational Challenges of Large Game Trees

- A real-time strategy game typically has a very large branching factor (many possible unit/action combinations per turn) and may need meaningful lookahead depth to anticipate real strategic consequences — since Minimax's basic time complexity is $O(b^d)$, even modest increases in either branching factor or depth make exhaustive evaluation computationally explosive.
- This is especially problematic given the scenario's "strict time limits" — the AI cannot simply be given unlimited thinking time before each decision, so raw exhaustive Minimax is not viable as-is.
- Memory can also become a concern if large parts of the tree must be held in memory during search, though the sheer number of node evaluations required is usually the more limiting factor in practice.

Task 3 — How Alpha-Beta Pruning Improves Efficiency, With Detailed Reasoning

Alpha-Beta pruning tracks two running bounds while traversing the tree: α (the best value the maximizing player can already guarantee) and β (the best value the minimizing player can already guarantee). Whenever, at some node, it becomes clear that a branch cannot possibly produce a result better than what the opponent could already force elsewhere in the tree ($\alpha \geq \beta$ at that point), the remaining branches under that node can be safely skipped entirely.

- This pruning never changes the final decision Minimax would have reached on its own — Alpha-Beta always returns exactly the same result as full Minimax, just faster — because the skipped branches are mathematically guaranteed not to affect the outcome; a rational opponent would simply never allow play to reach them.
- With good move ordering (evaluating the most promising moves first at each node), Alpha-Beta's best-case time complexity drops to $O(b^{(d/2)})$ — roughly allowing the AI to search twice as deep within the same time budget compared to plain Minimax, which is precisely the advantage needed under strict real-time limits.

Task 4 — Designing an Evaluation Function, and Justifying the Factors Considered

Since the full game tree cannot be explored to a terminal win/lose state within the time limit, Alpha-Beta must be depth-limited and rely on an evaluation function to score non-terminal states. A reasonable evaluation function for this real-time strategy game would combine several weighted factors:

- Material/unit balance — the AI's unit and structure strength minus the opponent's, a core measure of relative military strength.
- Resource advantage — stockpiled and incoming resources, since economic strength predicts future military strength.
- Map control/territory — the proportion of strategically important area (chokepoints, resource nodes) controlled.
- Positional safety — how exposed the AI's key units/base are to attack compared with the opponent's.
- Tempo/initiative — whether the AI currently has more productive options available, or the opponent has been forced onto the defensive.

Each factor is weighted and summed into a single numeric score, with weights typically tuned through playtesting or self-play, since relying on any single easily-gamed metric (e.g., raw unit count without accounting for unit strength or position) can produce a misleading evaluation.

Task 5 — Recommended Strategy for Balancing Optimality and Real-Time Performance

The recommended strategy is **Alpha-Beta pruning combined with iterative deepening**: run depth-limited Alpha-Beta search starting from a shallow depth, progressively increasing the depth limit, and stop as soon as the time budget for that decision is about to expire — returning the best move found at the deepest fully-completed depth.

- This guarantees the AI always has a legal, reasonably good move ready exactly when time runs out, rather than risking an incomplete search with no usable answer at a fixed target depth.
- Pairing this with good move ordering (e.g., trying the best move found in the previous, shallower iteration first) maximizes how much Alpha-Beta pruning actually occurs, pushing effective search depth as close as possible to the favourable $O(b^{(d/2)})$ case.
- Together, this balances optimality (searching as deep as time allows, with Alpha-Beta's exactness guarantee at whatever depth completes) against real-time performance (a hard time cutoff that always yields a usable decision on schedule) — the industry-standard approach in real-time game AI design.